

VLSI Internship Task 1 - Written Summary Report

1. Introduction to VLSI Design Flow

VLSI (Very Large Scale Integration) refers to the process of integrating millions or billions of transistors onto a single chip.

It is the foundation of modern electronics such as smartphones, laptops, embedded systems, and communication devices.

Steps in VLSI Design Flow:

1. Specification: Define chip functionality, speed, power, and area requirements.
2. Architecture Design: Create system-level structure and block diagrams.
3. RTL Design: Write hardware description using VHDL/Verilog.
4. Functional Verification: Verify logic behavior using simulation tools.
5. Logic Synthesis: Convert RTL into gate-level netlist.
6. Physical Design: Placement and routing of components.
7. Timing Analysis: Ensure timing constraints are met.
8. Fabrication: Semiconductor manufacturing of IC.
9. Testing and Packaging: Validate and package the final chip.

1.1 Specification

This is the starting point of chip design, where the requirements of the circuit or system are clearly defined.

What is decided here:

- **Functionality:** What should the chip do? (e.g., processor, memory, sensor interface, communication module)
- **Performance:** Required speed or operating frequency
- **Power consumption:** Maximum power budget, especially important for battery-powered devices

- **Area:** Maximum chip size/cost constraints
- **Technology node:** Whether design will use 180nm, 90nm, 45nm, 7nm, etc.

Example:

If designing a mobile processor, specifications may include:

- Clock speed = 2 GHz
- Power < 5W
- Support for 8 cores
- Cache size = 8 MB

Output of this stage:

- Design specification document
-

1.2. Architecture Design

After requirements are defined, the overall structure of the system is planned.

Tasks performed:

- Divide system into smaller blocks/modules
- Decide how modules communicate
- Create block diagrams
- Choose algorithms and data paths

Example modules:

- ALU (Arithmetic Logic Unit)
- Registers
- Memory blocks
- Control unit
- Input/output interfaces

Questions answered:

- How many processing units are needed?

- How will memory be organized?
- What buses or communication protocols are used?

Output:

- System architecture diagram
 - High-level design model
-

1.3. RTL Design (Register Transfer Level Design)

Here the architecture is converted into hardware description language code.

Languages used:

- **VHDL**
- **Verilog/SystemVerilog**

What happens:

- Describe logic behavior using code
- Define inputs, outputs, registers, and combinational logic

Example:

Instead of physically drawing gates, you write:

```
Y <= A AND B;
```

RTL focuses on:

- Data movement between registers
- Clock behavior
- Sequential and combinational logic

Output:

- RTL code files (.vhd, .v)
-

1.4. Functional Verification

This stage checks whether the RTL behaves according to the specification.

Goal:

Make sure design is logically correct **before manufacturing** (because fixing chips after fabrication is extremely expensive).

Methods:

- Simulation
- Testbenches
- Waveform analysis
- Assertion checking

Tools:

- ModelSim
- Vivado Simulator
- Cadence Xcelium

What is checked:

- Does output match expected values?
- Are corner cases handled?

Example:

For AND gate:

- $1 \text{ AND } 1 = 1$
- $1 \text{ AND } 0 = 0$

Output:

- Verified RTL design
 - Bug reports/fixes
-

1.5. Logic Synthesis

RTL code is converted into actual hardware gates.

Conversion:

RTL → Gate-level netlist

Tools:

- Synopsys Design Compiler
- Cadence Genus

What synthesis does:

- Converts HDL into standard cells
- Optimizes for:
 - Speed
 - Area
 - Power

Example:

`Y <= A XOR B;`

gets converted into a network of CMOS gates.

Output:

- Gate-level netlist

(Netlist = list of gates and their connections)

1.6. Physical Design

Now logical gates are physically placed onto silicon layout.

Main steps:

a) Floorplanning

- Decide placement of large blocks
- Define chip dimensions

b) Placement

- Place standard cells in layout

c) Clock Tree Synthesis (CTS)

- Distribute clock signal evenly

d) Routing

- Connect all gates with metal wires

Challenges:

- Congestion
- Delay
- Power distribution

Tools:

- Cadence Innovus
- Synopsys ICC2

Output:

- Physical layout of chip
-

1.7. Timing Analysis

Checks whether the chip can operate at target speed.

Key concept:

Signals must reach destination within clock cycle.

Important parameters:

- Setup time
- Hold time
- Propagation delay
- Clock skew

Tool:

- Static Timing Analysis (STA)

Example:

If clock period = 10 ns

Signal delay must be < 10 ns.

Problems checked:

- Timing violations
- Slow paths

Output:

- Timing closure report
-

1.8. Fabrication

Once design is finalized, chip is manufactured in semiconductor fabs.

Done in fabrication plants like:

- TSMC
- Intel
- Samsung Electronics

Process:

- Silicon wafer preparation
- Oxidation
- Photolithography
- Etching
- Ion implantation
- Deposition
- Metallization

This creates:

- Transistors
- Interconnect layers
- Contacts

Output:

- Manufactured silicon wafer with chips
-

1.9. Testing and Packaging

After fabrication, chips are tested and packaged.

Testing

Checks:

- Functional correctness
- Timing
- Power
- Manufacturing defects

Common tests:

- Scan testing
- Built-In Self-Test (BIST)
- Fault testing

Packaging

Chip is enclosed in protective package.

Examples:

- DIP
- QFP
- BGA

Why packaging matters:

- Protects chip
- Enables external connections
- Heat dissipation

Final output:

- Ready-to-use IC/chip

Example:

Microcontroller ready for PCB integration.

2. AND Gate Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

3. OR Gate Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

4. NOT Gate Truth Table

A	Y
0	1
1	0

5. NAND Gate Truth Table

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

6. NOR Gate Truth Table

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

7. XOR Gate Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

8. Half Adder Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

9. Full Adder Truth Table

A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

10. Observations from Simulation in Logisim

- All implemented logic gates produced outputs matching their theoretical truth tables.
- AND gate generated HIGH output only when both inputs were HIGH.
- OR gate generated HIGH when at least one input was HIGH.
- NOT gate inverted the input correctly.
- NAND and NOR gates worked as universal gates.
- XOR gate generated HIGH only when inputs were different.
- Half adder successfully produced correct Sum and Carry outputs.
- Full adder correctly handled carry input and generated accurate outputs.
- Logisim waveforms and circuit outputs matched expected results.